

PUSHING THE REDESIGN TO A NEW BRANCH

Step-by-step: connect this project folder to your existing GitHub repo and push this version on its own branch, without touching main.

Folder: `Desktop\ccd-web` Example branch: `ui/ayala-showcase-redesign`

Before you start: open a terminal **inside the project folder** (`C:\Users\Nicho Lescano\Desktop\ccd-web`). Git Bash or PowerShell both work. Replace anything in `<angle brackets>` with your own value. Nothing changes on GitHub until **step 10**, so steps 1–9 are safe to redo.

1 Turn this folder into a git repository

```
git init
```

Creates a hidden `.git` folder that tracks your files. Your files are not changed.

2 Connect it to your repo on GitHub

```
git remote add origin <your-repo-url>
```

`origin` is the usual nickname for "the repo on GitHub". Check it worked:

```
git remote -v
```

3 Download your repo's history

```
git fetch origin
```

Downloads what's on GitHub without touching the files in your folder.

4 Find your main branch's name

```
git branch -r
```

You'll see something like `origin/main` or `origin/master`. Use that name in step 6.

5 Create your new branch

```
git checkout -b ui/ayala-showcase-redesign
```

`-b` means "create a new branch and switch to it". You can use any name you like.

6 Base the branch on your repo's main branch

```
git reset origin/main
```

Use `origin/master` if that's what step 4 showed. This points your branch at the latest commit on GitHub but **keeps all your current files exactly as they are**. Git now sees the redesign as changes on top of main, so you can open a pull request later.

7 Check what changed before saving anything

```
git status
```

- **Modified / untracked** files are your redesign work.
- **Deleted** files exist in your repo but not in this folder. Look at these carefully. To keep one, restore it with:

```
git checkout origin/main -- <path/to/file>
```

- **.env must NOT be listed**, because it holds your passwords and keys. Confirm it's ignored (this should print `.env`):

```
git check-ignore .env
```

8 Stage everything

```
git add -A
```

Marks every change, including deletions, to go into the commit.

9 Save it as a commit

```
git commit -m "Redesign screenings showcase, film page and booking flow"
```

If git asks who you are (first time only), run these two, then repeat the commit:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

10 Push the new branch to GitHub

```
git push -u origin ui/ayala-showcase-redesign
```

`-u` links your local branch to the one on GitHub, so next time a plain `git push` is enough. Git may open a browser window or ask you to sign in.

11 Open a pull request (optional)

After the push, git prints a link like `https://github.com/.../pull/new/ui/ayala-showcase-redesign`. Open it to review your changes against main before merging.

If something goes wrong

"remote origin already exists" in step 2: run this instead:

```
git remote set-url origin <your-repo-url>
```

Step 7 shows a lot of deletions you didn't expect: stop and don't commit. Your repo probably has files or folders this copy doesn't. Ask for help before going further.

Nothing has reached GitHub yet: everything before step 10 is local only, so you can start over safely.